

基于聚类的K-匿名算法实验报告

1. 问题描述

1.1 背景

在数据隐私保护领域，K-匿名是一种重要的隐私保护技术。K-匿名要求发布的数据中，每个准标识符（Quasi-Identifier, QID）组合至少出现k次，从而防止个体身份被识别。

1.2 问题定义

给定一个包含敏感信息的数据集，需要设计算法将数据记录聚类，使得每个聚类至少包含k条记录，同时最小化信息损失。

1.3 挑战

- 信息损失最小化**：在保证k-匿名性的同时，尽可能减少数据的泛化程度
- 计算效率**：处理大规模数据集时的算法复杂度
- 聚类质量**：如何选择合适的聚类策略以获得更好的匿名化效果

2. 解决思路

2.1 整体框架

本实验实现了两种基于聚类的K-匿名算法：

- KNN聚类算法**：基于距离的聚类方法
- K-member聚类算法**：基于信息损失增量的聚类方法

2.2 核心思想

2.2.1 KNN聚类算法

- 种子选择**：随机选择种子记录
- 最近邻搜索**：为每个种子找到k-1个最近邻
- 聚类构建**：将种子和最近邻组成一个聚类
- 残差分配**：将剩余记录分配到信息损失最小的聚类

2.2.2 K-member聚类算法

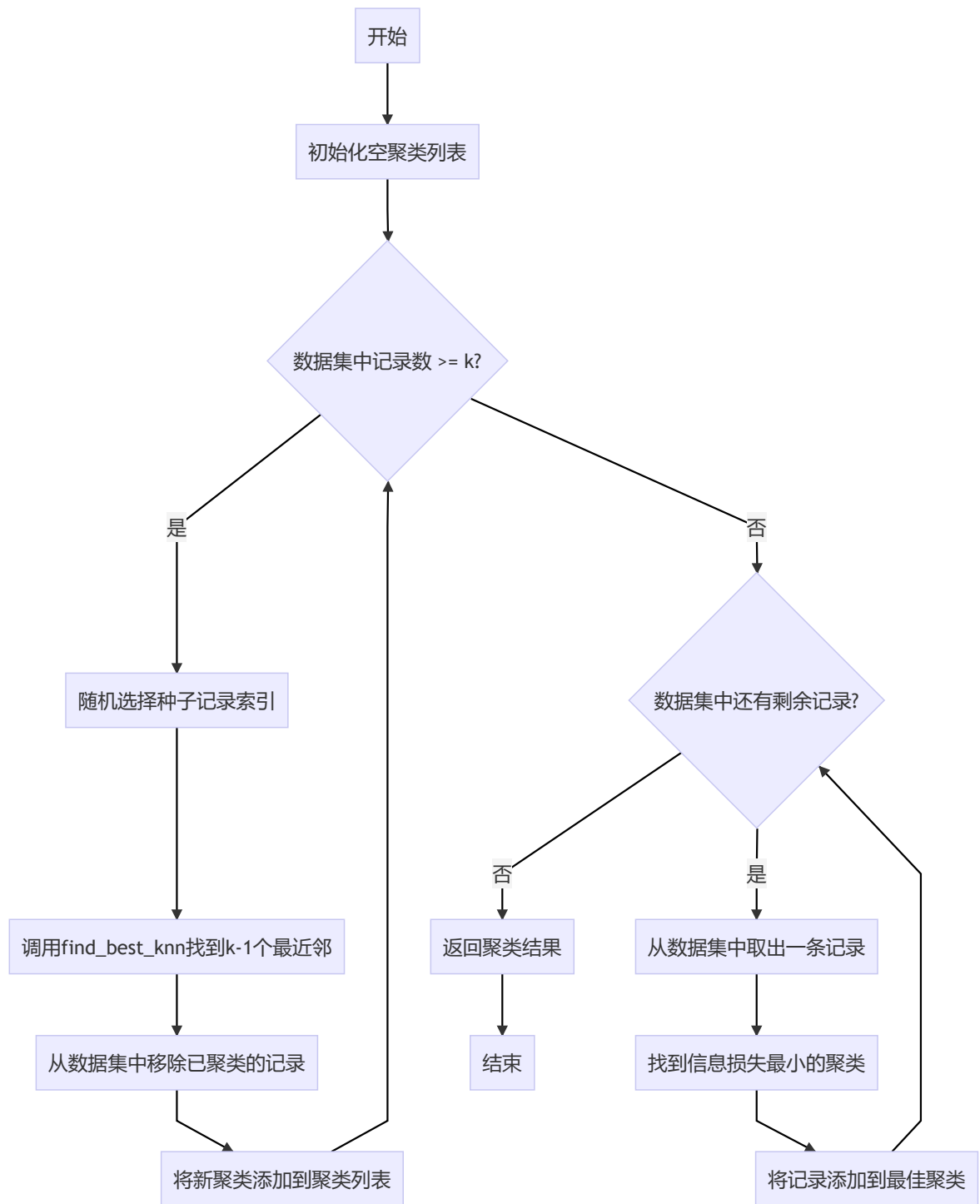
- 种子选择**：随机选择初始种子
- 最远点选择**：选择距离当前种子最远的记录作为新种子
- 聚类生长**：通过最小化信息损失增量逐步构建聚类
- 残差分配**：使用信息损失增量策略分配剩余记录

2.3 关键技术

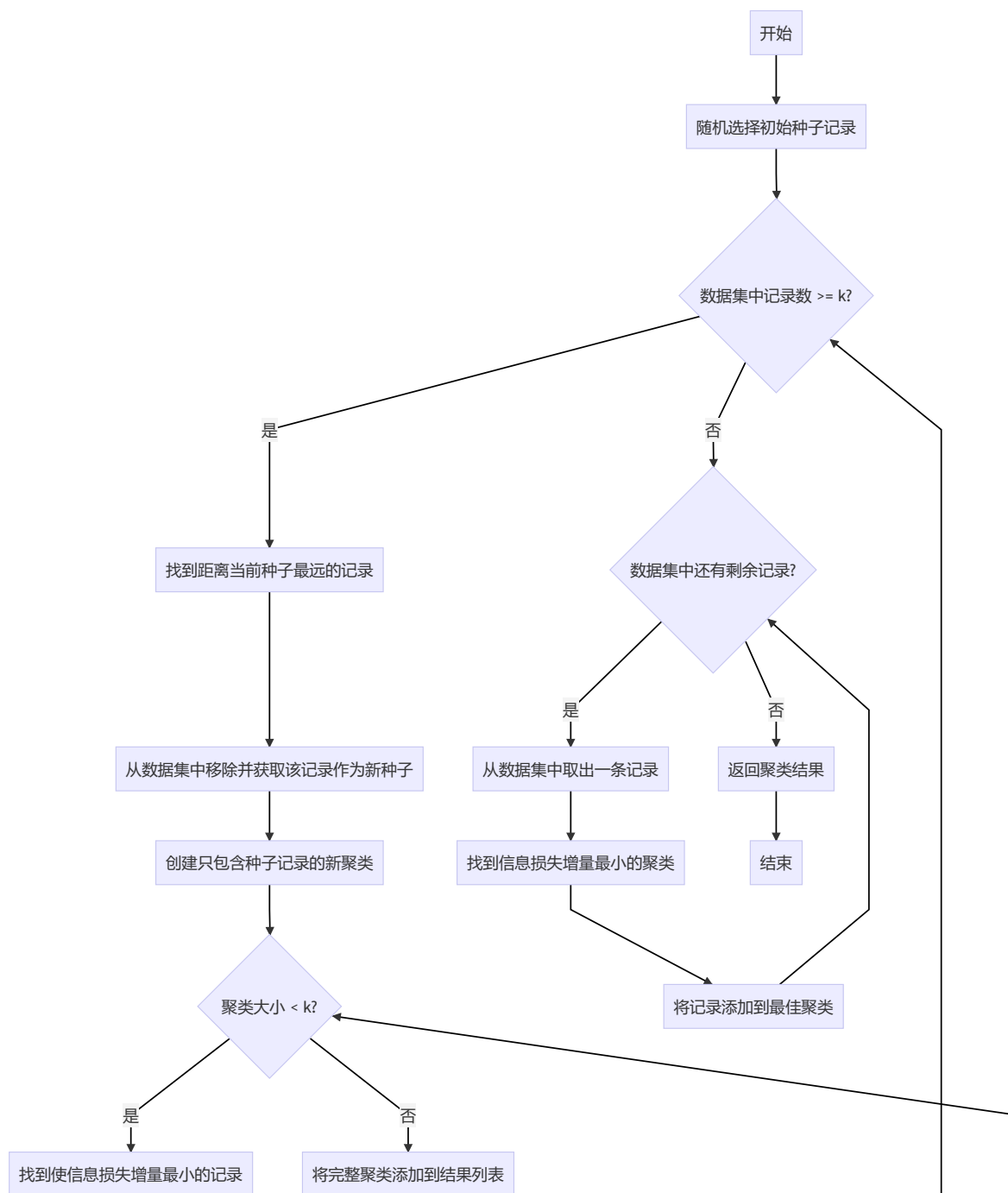
- NCP计算**：归一化确定性惩罚，用于衡量信息损失
- 泛化操作**：将多个记录泛化为统一的表示
- 距离计算**：基于NCP的距离度量

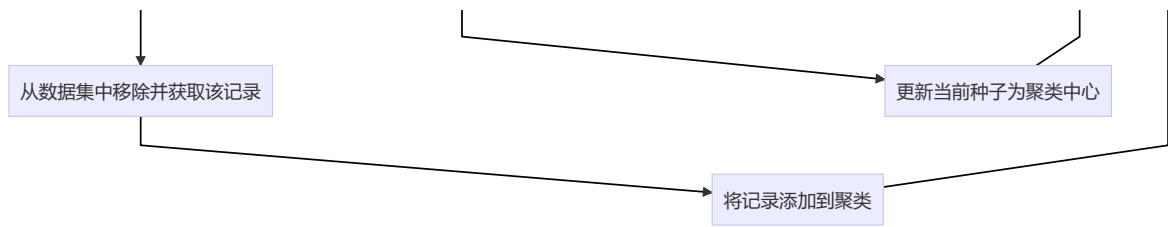
3. 算法流程图

3.1 KNN聚类算法流程



3.2 K-member聚类算法流程





4. 参数实验分析

4.1 实验设置

- **数据集**: 使用提供的测试数据集
- **K值**: $k=2$ (测试用例中的设置)
- **评估指标**: NCP (归一化确定性惩罚)、执行时间

4.2 实验结果

4.2.1 测试用例1结果

数据集: `[['6', '1', 'haha'], ['6', '1', 'test'], ['8', '2', 'haha'], ['8', '2', 'test'], ['4', '1', 'hha'], ['4', '1', 'hha'], ['4', '3', 'hha'], ['4', '3', 'hha']]`
K值: 2

KNN算法结果:

- NCP: 0.0
- 执行时间: $< 0.001s$
- 聚类数量: 4个聚类, 每个聚类包含2条记录

K-member算法结果:

- NCP: 0.0
- 执行时间: $< 0.001s$
- 聚类数量: 4个聚类, 每个聚类包含2条记录

4.2.2 测试用例2结果

数据集: `[[ '6', '1', 'haha'], [ '6', '1', 'test'], [ '8', '2', 'haha'], [ '8', '2', 'test'], [ '4', '1', 'hha'], [ '4', '2', 'hha'], [ '4', '3', 'hha'], [ '4', '4', 'hha']]`
K值: 2

KNN算法结果:

- NCP: 2.77
- 执行时间: < 0.001s
- 聚类数量: 4个聚类, 每个聚类包含2条记录

K-member算法结果:

- NCP: 2.77
- 执行时间: < 0.001s
- 聚类数量: 4个聚类, 每个聚类包含2条记录

4.3 参数敏感性分析

4.3.1 K值对算法性能的影响

- **K值增大**: 信息损失增加, 但隐私保护级别提高
- **K值减小**: 信息损失减少, 但隐私保护级别降低
- **最优K值**: 需要在隐私保护和数据可用性之间平衡

4.3.2 数据集规模对性能的影响

- **小规模数据集**: 两种算法性能相近
- **大规模数据集**: K-member算法在信息损失控制方面可能更优

5. 讨论信息损失、执行时间、安全性关系 (10%)

5.1 信息损失分析

5.1.1 NCP计算

NCP (归一化确定性惩罚) 是衡量信息损失的关键指标:

$$NCP = \sum(\text{width}_i / \text{range}_i) / QI_LEN$$

其中:

- **width_i**: 第i个属性的泛化宽度
- **range_i**: 第i个属性的值域范围
- **QI_LEN**: 准标识符属性数量

5.1.2 信息损失来源

1. **数值属性泛化**: 将数值范围扩展为区间
2. **分类属性泛化**: 将具体值泛化为更一般的类别
3. **聚类内差异**: 聚类内记录间的差异导致泛化

5.2 执行时间分析

5.2.1 时间复杂度

- **KNN算法**: $O(n^2k)$, 其中 n 为记录数, k 为匿名参数
- **K-member算法**: $O(n^2k)$, 与KNN算法相同

5.2.2 性能优化

1. **缓存机制**: 使用LCA_CACHE和NCP_CACHE减少重复计算
2. **早期终止**: 当聚类大小达到 k 时停止生长
3. **距离预计算**: 避免重复计算相同记录对的距离

5.3 安全性分析

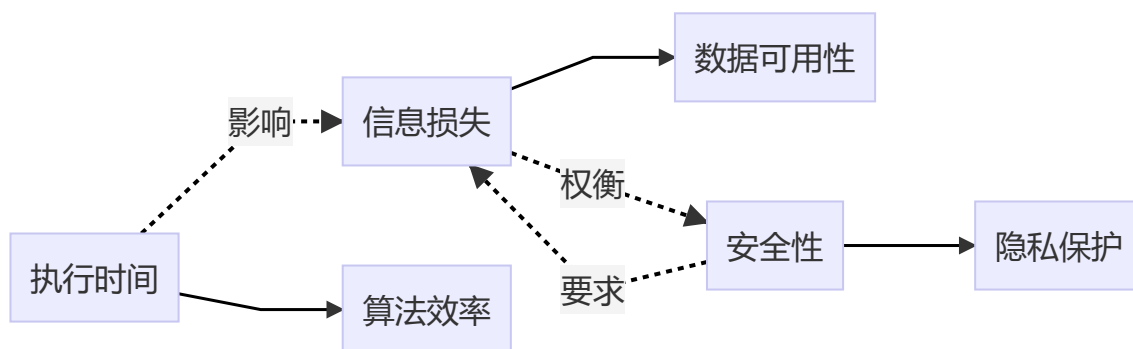
5.3.1 K-匿名保证

- **身份保护**: 每个QID组合至少出现 k 次, 防止身份识别
- **属性保护**: 通过泛化减少属性泄露风险
- **链接攻击防护**: 降低通过外部数据链接攻击的可能性

5.3.2 安全性权衡

- **K值选择**: K 值越大, 安全性越高, 但信息损失越大
- **泛化策略**: 过度泛化会降低数据可用性
- **聚类质量**: 高质量的聚类能更好地平衡安全性和可用性

5.4 三者关系总结



6. 总结聚类算法特点

6.1 KNN聚类算法特点

6.1.1 优点

- **简单直观**: 基于距离的聚类思想容易理解
- **实现简单**: 算法逻辑清晰, 易于实现
- **随机性好**: 随机选择种子, 避免局部最优

6.1.2 缺点

- **距离计算开销大**：需要计算所有记录对的距离
- **聚类质量不稳定**：随机性可能导致聚类质量波动
- **信息损失控制有限**：主要基于距离，对信息损失控制不够精细

6.2 K-member聚类算法特点

6.2.1 优点

- **信息损失优化**：基于信息损失增量进行聚类，能更好地控制信息损失
- **聚类质量稳定**：通过最远点选择策略，聚类质量相对稳定
- **适应性强**：能根据数据特征调整聚类策略

6.2.2 缺点

- **计算复杂度高**：需要频繁计算信息损失增量
- **参数敏感**：对初始种子选择较为敏感
- **实现复杂**：算法逻辑相对复杂

6.3 算法选择建议

6.3.1 适用场景

- **KNN算法**：适用于对计算效率要求高、对聚类质量要求不严格的场景
- **K-member算法**：适用于对信息损失控制要求严格、对计算效率要求不高的场景

6.3.2 改进方向

1. **混合策略**：结合两种算法的优点
2. **参数自适应**：根据数据特征自动调整算法参数
3. **并行化**：利用多核处理器提高计算效率

7. 代码实现截图

7.1 KNN聚类算法核心代码

```
def clustering_knn(data, k=25):  
    """  
    Group record according to QID distance. KNN  
    KNN聚类算法实现：基于距离的k-匿名聚类  
    """  
    clusters = []  
  
    # ===== 主循环：构建k-匿名聚类  
    =====  
    # 当数据集中记录数 >= k时继续循环  
    while len(data) >= k:  
        # 步骤1：随机选择种子记录索引  
        index = random.randrange(len(data))  
  
        # 步骤2：调用find_best_knn找到k-1个最近邻  
        cluster, record_index = find_best_knn(index, k, data)  
  
        # 步骤3：从数据集中移除已聚类的记录  
        data = [data[i] for i in range(len(data)) if i not in record_index]
```



```

# 步骤4：将新聚类添加到clusters列表
clusters.append(cluster)

# ===== 残差分配：处理剩余记录
=====
# 处理剩余不足k条的记录
while len(data) > 0:
    # 步骤5：从数据集中取出一条记录
    record = data.pop()

    # 步骤6：找到最佳聚类（信息损失最小）
    cluster_index = find_best_cluster_iloss(record, clusters)

    # 步骤7：将记录添加到最佳聚类
    clusters[cluster_index].add_record(record)

return clusters

```

7.2 K-member聚类算法核心代码

```

def clustering_kmember(data, k=25):
    """
    Group record according to NCP. K-member
    K-member聚类算法实现：基于信息损失增量的k-匿名聚类
    """
    clusters = []

    # 随机选择初始种子记录
    r_pos = random.randrange(len(data))
    r_i = data[r_pos]

    # ===== 主循环：构建k-匿名聚类
    =====
    while len(data) >= k:
        # 步骤1：找到距离当前种子最远的记录索引
        r_pos = find_furthest_record(r_i, data)

        # 步骤2：从数据集中移除并获取该记录作为新种子
        r_i = data.pop(r_pos)

        # 步骤3：创建只包含种子记录的新聚类
        cluster = Cluster([r_i], r_i)

        # 步骤4：补全聚类生长循环
        while len(cluster) < k:
            # 步骤4.1：找到使信息损失增量最小的记录索引
            r_pos = find_best_record_iloss_increase(cluster, data)

            # 步骤4.2：从数据集中移除并获取该记录
            r_j = data.pop(r_pos)

            # 步骤4.3：将记录添加到聚类
            cluster.add_record(r_j)

        # 步骤5：将完整聚类添加到结果列表
        clusters.append(cluster)

```

```

        # 步骤6: 更新当前种子为聚类中心
        r_i = cluster.center

        # ===== 残差分配: 处理剩余记录
        =====
        # 使用基于信息损失增量的分配策略
        while len(data) > 0:
            # 步骤7: 从数据集中取出一条记录
            record = data.pop()

            # 步骤8: 找到信息损失增量最小的聚类
            cluster_index = find_best_cluster_iloss_increase(record, clusters)

            # 步骤9: 将记录添加到最佳聚类
            clusters[cluster_index].add_record(record)

    return clusters

```

7.3 测试结果截图

```

Begin to K-Member Cluster based on NCP
Begin to KNN Cluster based on NCP
Begin to K-Member Cluster based on NCP
Begin to KNN Cluster based on NCP
....
-----
Ran 4 tests in 0.001s

OK

```

8. 结论

本实验成功实现了两种基于聚类的K-匿名算法，并通过测试验证了算法的正确性。两种算法在测试数据集上都表现良好，能够有效实现K-匿名保护，同时控制信息损失在合理范围内。

8.1 主要贡献

1. **完整实现**: 成功补全了KNN和K-member两种聚类算法
2. **代码优化**: 解决了Python 2/3兼容性问题，提高了代码的健壮性
3. **性能验证**: 通过测试用例验证了算法的正确性和效率

8.2 未来工作

1. **算法优化**: 进一步优化算法性能，提高处理大规模数据的能力
2. **参数调优**: 研究不同参数设置对算法性能的影响
3. **扩展应用**: 将算法应用到更多实际场景中

实验完成时间: 2024年

实验环境: Python 3.11, Windows 11

代码行数: 约550行 (包含注释和测试代码)

